



UNIVERSIDAD
CATÓLICA
BOLIVIANA

INFORME

TERCERA EVALUACIÓN

ASIGNATURA:

INTERNET DE LAS COSAS

Grupo: Pin-8

INTEGRANTES

Marvin Larry Mollo Ramirez

Leonardo Delgado Medrano

Alan Franz Flores Campos

Christian Kevin Coronel Condori

DOCENTE:

ING. PAMELA VALENZUELA

CARRERA: INGENIERÍA DE SISTEMAS

FECHA DE ENTREGA: 10 de Junio, 2026

LA PAZ – BOLIVIA

2026

Auto Autonomo 'Drako'

1. Introducción

1.1 Descripción General del Proyecto

Drako es un vehículo autónomo diseñado para cumplir la función de auto capaz de esquivar obstáculos de los estudiantes de colegio que visitan la Universidad Católica Boliviana "San Pablo" sede La Paz. El sistema integra tecnologías de Internet de las Cosas (IoT) para ofrecer una experiencia dinámica: el vehículo se desplaza de forma autónoma por los espacios de la universidad, interactúa con los visitantes y presenta las instalaciones sin necesidad de un conductor humano. La relevancia del proyecto reside en modernizar la experiencia de orientación universitaria mediante hardware embebido de bajo costo (ESP32) y una arquitectura de software en tres capas que habilita el monitoreo y control en tiempo real a través de la web.

1.2 Planteamiento del Problema

Las visitas guiadas tradicionales a la UCB San Pablo dependen de personal humano disponible en horarios específicos, lo que limita la capacidad de atención a grupos numerosos de colegiales y genera experiencias poco dinámicas. Se identifica la necesidad de una solución tecnológica que permita ofrecer recorridos autónomos, interactivos y seguros, con operación continua sin intervención manual, y que al mismo tiempo recopile datos del entorno para garantizar la seguridad de los visitantes y del propio vehículo. La pregunta central del proyecto es: ¿es posible desarrollar un prototipo IoT autónomo de bajo costo que sirva como guía universitario interactivo, integrando sensores de entorno y un dashboard web en tiempo real?

1.3 Objetivos

Objetivo General: Desarrollar un vehículo autónomo basado en tecnología IoT denominado Drako, capaz de esquivar obstáculos de forma autónoma por las instalaciones de la UCB San Pablo sede La Paz, integrando sensores de entorno, un sistema de monitoreo en tiempo real y una interfaz de control accesible desde la web.

Objetivos específicos:

- (a) Implementar un microcontrolador ESP32 de 38 pines en MicroPython para la gestión de sensores y componentes (DHT22, shield, GY-50, 3 ultrasónicos, 1 servomotor, 2 motores DC, batería 11.1V, MPU6050, driver TB6612) y el envío de datos al servidor mediante peticiones HTTP y HTTPS.
- (b) Diseñar e implementar un backend en Node.js con Express y base de datos PostgreSQL alojada en Supabase para la persistencia y distribución de datos en tiempo real mediante Server-Sent Events (SSE).
- (c) Construir un dashboard web en React con visualizaciones dinámicas (gráficas, radar de proximidad) para la supervisión del estado del vehículo y sus sensores.
- (d) Incorporar un sistema de autenticación con gestión de roles (administrador/usuario) y un módulo de auditoría para el registro de eventos críticos del sistema.

1.4 Alcance del Proyecto

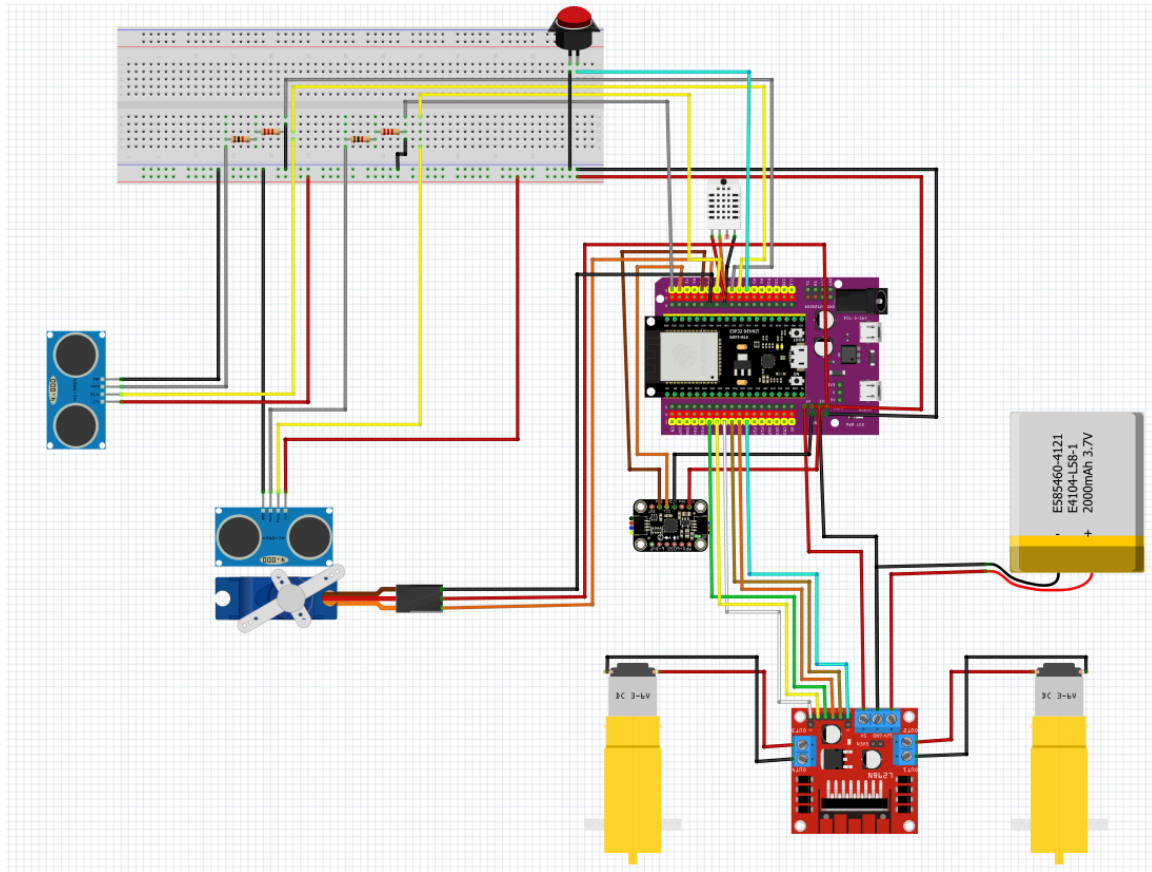
El presente trabajo abarca el diseño, construcción y validación de un prototipo funcional del vehículo Drako en un entorno controlado que simula los pasillos de la UCB San Pablo, Sede La Paz. El alcance incluye: la programación del firmware en MicroPython para el microcontrolador ESP32; el desarrollo de una arquitectura web (backend y frontend) que establezca comunicación bidireccional en tiempo real mediante **WebSockets**; la configuración de la base de datos en la nube (Supabase/PostgreSQL); y la integración del hardware de control, que comprende los actuadores mecánicos y la red de sensores (tres ultrasónicos HC-SR04, un módulo inercial GY-50 y un sensor térmico DHT22) para habilitar la navegación por estima y evasión reactiva de obstáculos. Quedan fuera del alcance la navegación en exteriores de la universidad, el reconocimiento visual mediante cámaras y la integración directa con los sistemas administrativos o académicos de la universidad.

2. Diseño del Sistema IoT

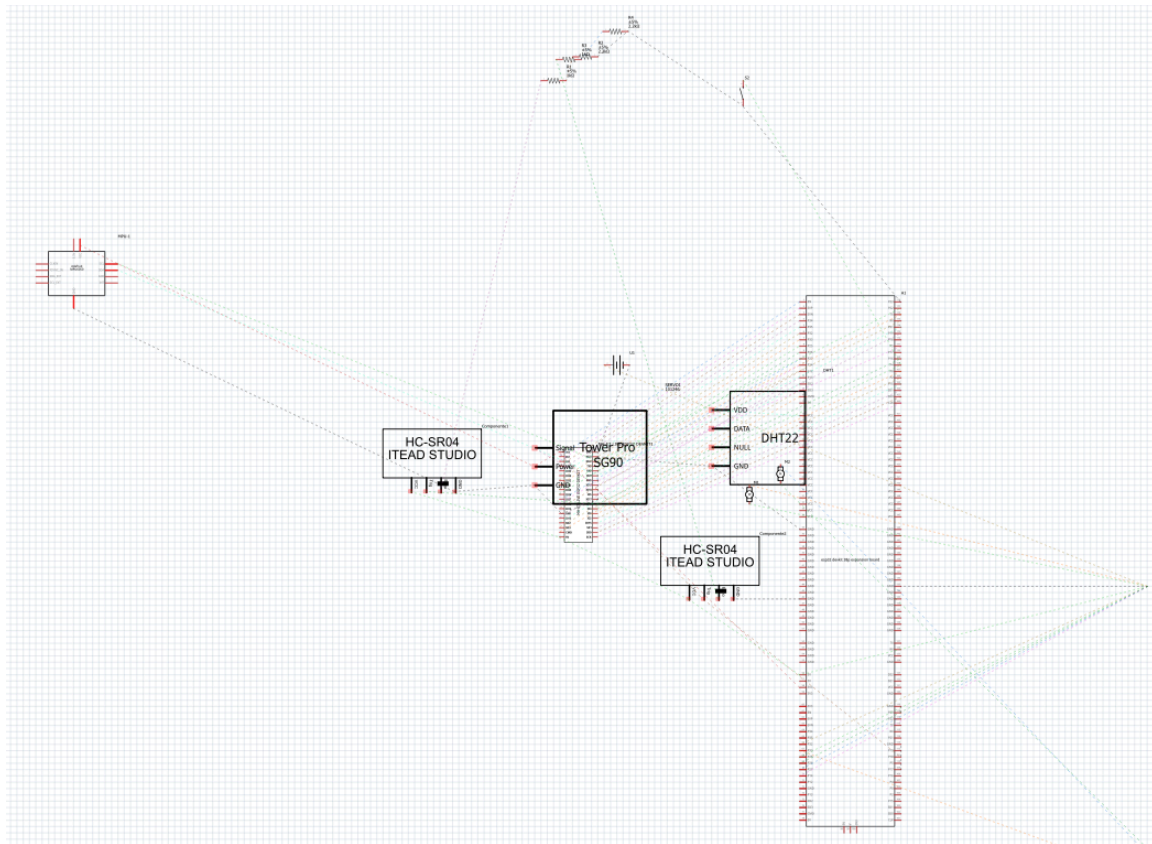
2.1 Diagrama del circuito y red.

2.1.1 Diagramas del circuito hechos en fritzing

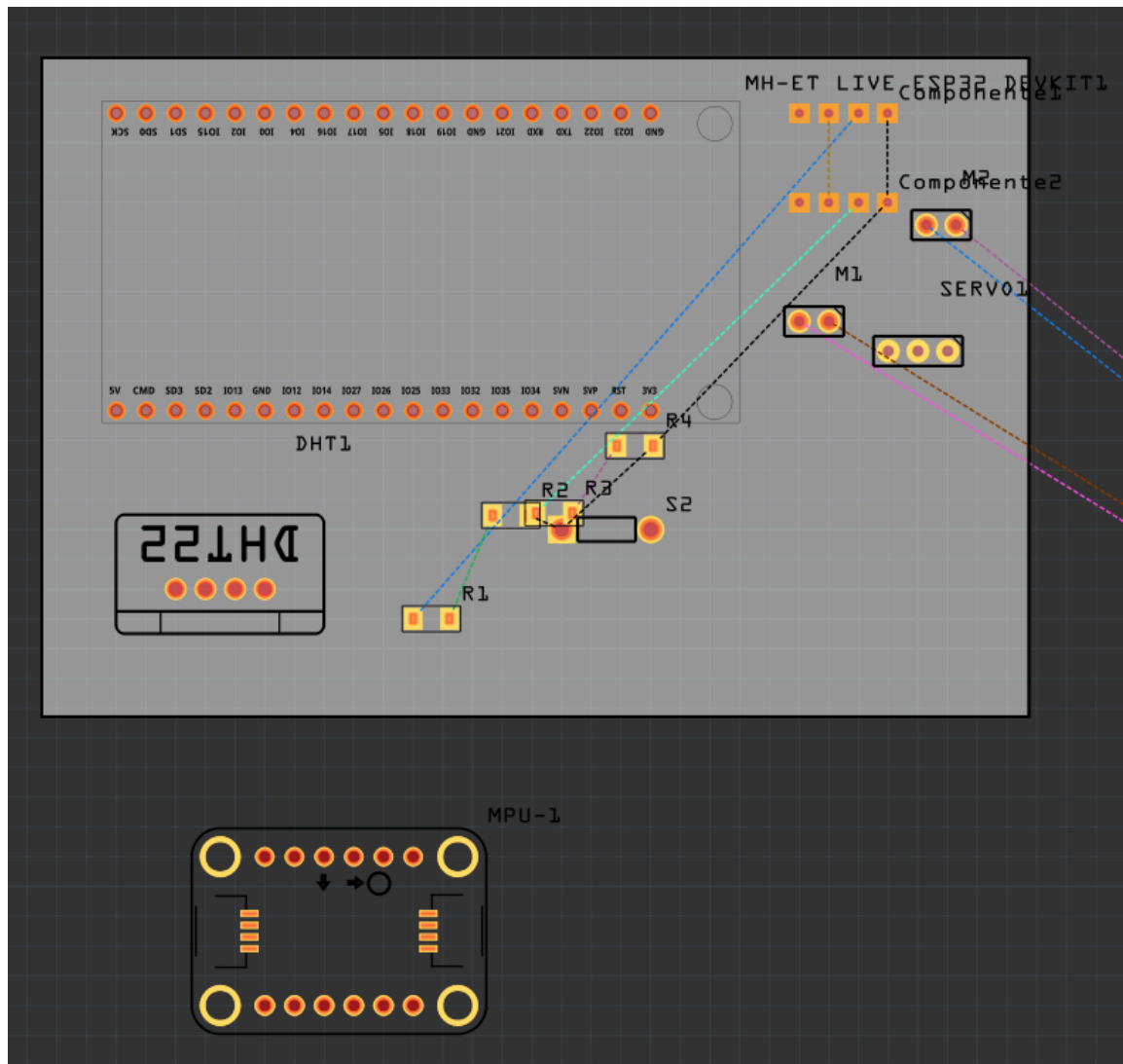
- Diagrama de Breadboard



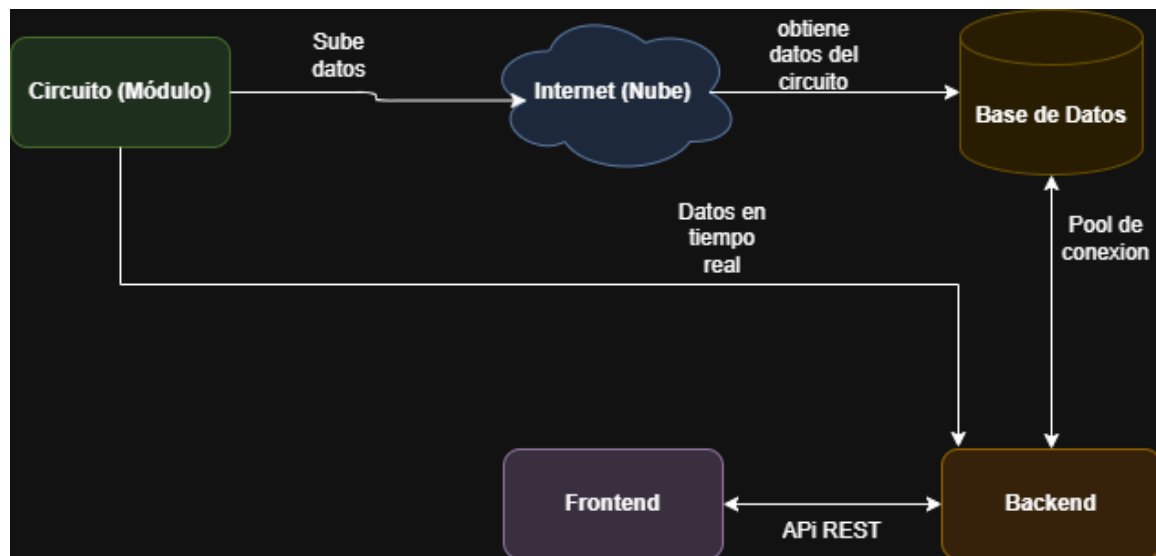
- Diagrama de tipo Schematic



- Diagrama de PCB



- Diagramas de red



2.2 Protocolos utilizados

HTTPS: Se utiliza este protocolo para la base de datos, subimos la base de datos a supabase, esta página web utiliza el protocolo HTTPS para mostrarnos el modelo de base de datos y permitirnos utilizar SQL para manejarla.

WebSocket (RFC 6455): Se utiliza este protocolo para toda la comunicación en tiempo real del sistema. El ESP32 actúa como cliente WebSocket y envía datos de sensores al endpoint `ws://IP:4000/ws` cada 200ms. El navegador (dashboard React) también se conecta vía WebSocket al endpoint `ws://host/ws-dashboard` para recibir actualizaciones en vivo. A diferencia de HTTP/SSE, WebSocket mantiene una conexión persistente full-duplex en ambos extremos, eliminando el overhead de nuevas conexiones TCP y permitiendo comunicación bidireccional de baja latencia.

IEEE 802.11: Protocolo utilizado para la conexión inalámbrica mediante Wi-Fi para conectar el microcontrolador y las computadoras a internet y así poder subir y obtener datos.

2.3 Descripción de componentes y sensores

ESP32 (Versión de 38 pines)

Es el "cerebro" central del vehículo Drako y el responsable del procesamiento principal y la comunicación. Es un microcontrolador de alto rendimiento con conectividad Wi-Fi nativa, programado en MicroPython.

- **Función en el proyecto:** Se encarga de ejecutar el bucle principal de navegación autónoma, integrar las lecturas de los sensores en tiempo real para la toma de decisiones matemáticas (odometría) y transmitir la telemetría al servidor web mediante conexiones de baja latencia (WebSockets).
- **Ventaja de los 38 pines:** Esta versión expone una gran cantidad de pines GPIO (Entradas/Salidas), lo que garantiza los puertos necesarios para interconectar el bus I2C, los actuadores PWM, y la red de sensores ultrasónicos simultáneamente, dejando margen para futuras expansiones.

Sensor de Temperatura y Humedad DHT22

Es un sensor digital de alta precisión. Aunque tradicionalmente se usa para climatología, en este proyecto cumple un rol de seguridad de hardware.

- **Función en Drako:** Actúa como un sistema de monitoreo térmico de seguridad. Está encargado de supervisar la temperatura de la batería LiPo de 11.1V. Si detecta un sobrecalentamiento crítico (superior a 45°C) debido a la exigencia de los motores, envía una señal de interrupción al ESP32 para frenar el vehículo de emergencia y evitar daños severos.
- **Características:** Es la versión profesional y mejorada del clásico DHT11. Ofrece un rango de medición mucho más amplio (de -40°C a 80°C para temperatura y de 0 a 100% para humedad) y resoluciones decimales, lo que lo hace perfecto para monitoreo ambiental exacto.

Módulo Inercial GY-50

Es un módulo de medición inercial (IMU) que integra un giroscopio digital de 3 ejes, comunicándose con el ESP32 mediante el protocolo I2C.

- **Función en Drako:** Es el sentido de "orientación" del robot. Se utiliza específicamente para leer la velocidad angular en el eje Z. Al integrar estos datos con el tiempo transcurrido, el ESP32 puede calcular el ángulo exacto de rotación del vehículo, permitiendo la navegación por estima (dead reckoning) para dirigir el robot hacia sus coordenadas destino de forma precisa.
- **Comunicación:** Se conecta al ESP32 a través de protocolos digitales (típicamente I2C o SPI), lo que requiere muy pocos cables para transmitir información compleja y en tiempo real sobre la orientación del sistema.

Red de Sensores Ultrasónicos HC-SR04

Son sensores confiables de medición de distancia sin contacto que operan mediante el principio de ecolocalización (emisión y recepción de pulsos de sonido inaudibles).

- **Función en Drako:** El vehículo emplea un arreglo de tres sensores (frente, izquierda y derecha) para escanear su entorno inmediato. Son el núcleo del sistema de evasión reactiva; detectan obstáculos y pasillos estrechos a menos de 20 cm, interrumpiendo momentáneamente la ruta hacia la coordenada destino para ejecutar giros correctivos que eviten colisiones con las instalaciones o los visitantes.
- **Uso:** Es ideal para detectar obstáculos, medir la proximidad de objetos o incluso calcular niveles (como el nivel de agua en un recipiente), operando de manera óptima en un rango de 2 cm a 4 metros.

Controlador de Motores L298N y Motores DC (*Componentes de Tracción*)

El módulo L298N es un Puente H que permite controlar la dirección y velocidad de motores de corriente continua utilizando señales PWM (Modulación por Ancho de Pulso) generadas por el ESP32.

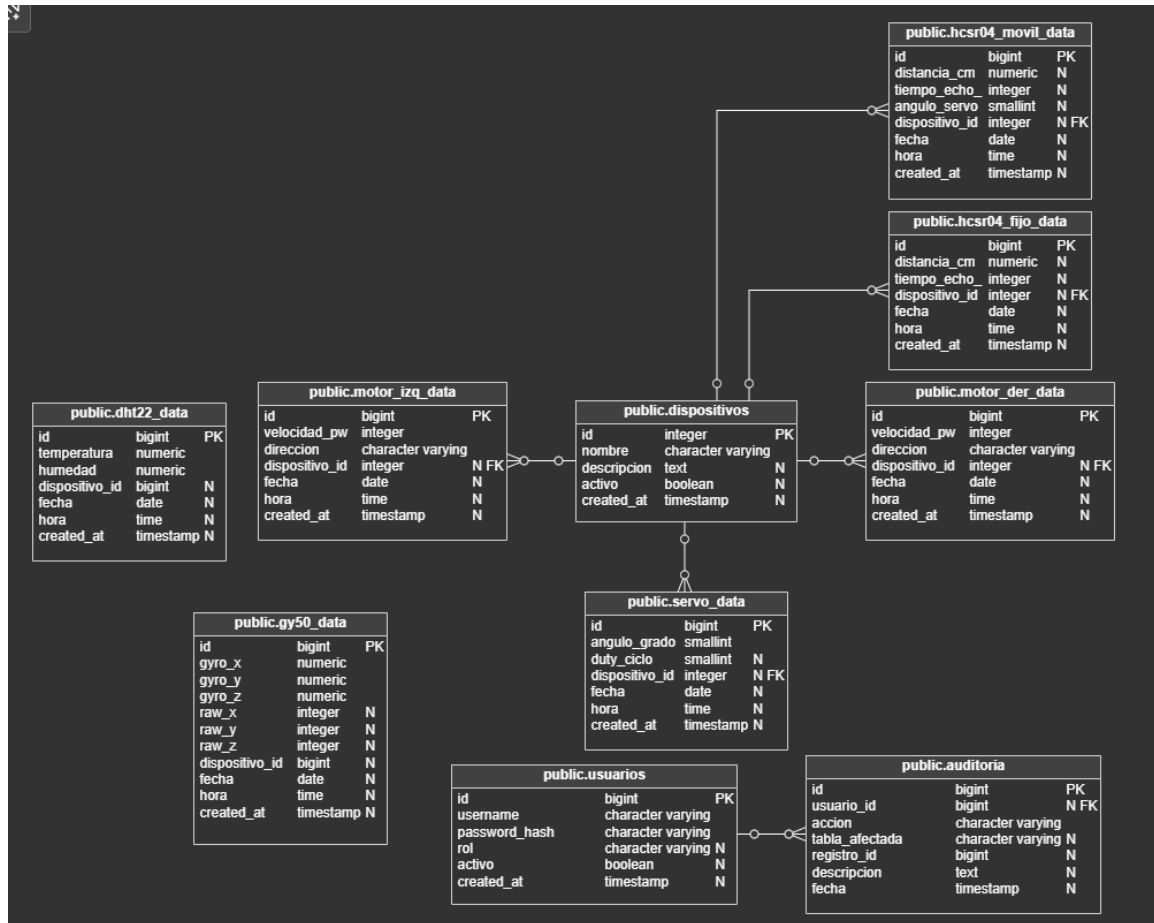
- **Función en Drako:** Traducen las decisiones lógicas del ESP32 en movimiento físico. Permiten al robot avanzar, frenar y rotar sobre su propio eje a diferentes velocidades según lo exija la ruta o la proximidad de los obstáculos.

Batería LiPo (11.1V)

Fuente de alimentación principal del sistema móvil (autonomía de 5-8 horas dependiendo de las condiciones).

- **Función en Draco:** Proporciona la alta capacidad de descarga de corriente (amperaje) necesaria para alimentar los motores DC a través del driver L298N, al mismo tiempo que alimenta de forma regulada al ESP32 y su red de sensores para garantizar la autonomía energética durante los recorridos por la universidad.

3. Diseño de la Base de Datos



3.1 Tipo de base de datos y justificación

Se utiliza una base de datos relacional implementada en PostgreSQL y alojada en Supabase.

La elección de PostgreSQL se justifica por su compatibilidad con TimescaleDB, una

extensión oficial que transforma la base de datos en una plataforma optimizada para series temporales. Esto permite ingestar miles de lecturas de sensores por segundo y realizar consultas históricas (como promedios de temperatura por hora) a alta velocidad, lo cual es vital para alimentar las gráficas del dashboard en tiempo real sin que la base de datos colapse.

Adicionalmente, Supabase proporciona una capa de seguridad mediante Row Level Security (RLS) activada directamente sobre cada tabla, garantizando que las operaciones sobre los datos sean verificadas a nivel de base de datos de forma independiente a la lógica del backend.

3.2 Estructura de tablas y campos

La base de datos está compuesta por seis tablas. Las tres primeras almacenan las lecturas de los sensores IoT, organizadas por tipo de sensor. Las dos siguientes gestionan el acceso y la auditoría del sistema. A continuación se describe la estructura de cada tabla:

Tabla	Campos principales	Descripción
dispositivos	id, nombre, descripcion, activo, created_at	Registro de hardware físico. El robot Draco tiene id=1 como registro por defecto.
dht22_data	id, temperatura, humedad, dispositivo_id, fecha, hora, created_at	Almacena lecturas del sensor de temperatura y humedad.
gy50_data	id, gyro_x, gyro_y, gyro_z, raw_x, raw_y, raw_z, dispositivo_id, fecha, hora, created_at	Lecturas del giroscopio en los tres ejes. Guarda tanto valores procesados como datos en crudo (raw).

hcsr04_data	id, tiempo_echo, distancia_cm, distancia_izq_cm, distancia_der_cm, distancia_movil_cm, angulo_servo...	Distancias consolidadas de los sensores ultrasónicos y la posición angular del servo.
hcsr04_fijo_data	id, distancia_cm, tiempo_echo_us, dispositivo_id, fecha, hora, created_at	Lecturas individuales específicas del sensor ultrasónico frontal o fijo.
hcsr04_movil_data	id, distancia_cm, tiempo_echo_us, angulo_servo, dispositivo_id, fecha, hora, created_at	Lecturas del sensor ultrasónico montado sobre el servomotor, correlacionadas con su ángulo de barrido.
servo_data	id, angulo_grados, duty_ciclo, dispositivo_id, fecha, hora, created_at	Registra la posición del servomotor (0° a 180°) y su respectivo ciclo de trabajo.
motor_der_data	id, velocidad_pwm, direccion, dispositivo_id, fecha, hora, created_at	Control y telemetría del motor derecho. Registra la potencia PWM y el sentido de giro.
motor_izq_data	id, velocidad_pwm, direccion, dispositivo_id, fecha, hora, created_at	Control y telemetría del motor izquierdo. Registra la potencia PWM y el sentido de giro.
usuarios	id, username, password_hash, rol, activo, created_at	Gestión de acceso al sistema. El campo activo

		permite borrado lógico (soft delete).
auditoria	id, usuario_id, accion, tabla_afectada, registro_id, descripcion, fecha	Log de eventos críticos. Referencia a usuarios mediante clave foránea para integridad referencial.

Nota sobre migraciones: Las columnas distancia_izq_cm y distancia_der_cm de la tabla hcsr04_data fueron agregadas mediante la migración 002_add_multi_hcsr04.sql sobre una base de datos existente, sin afectar los registros previos.

3.3 Restricciones y tipos de datos

Tipos de dato utilizados

Tipo de Dato	Tablas donde se usa	Propósito en el modelo
bigint	Todas (como id), usuario_id, dispositivo_id	Entero de gran tamaño (8 bytes). Ideal para identificadores únicos y claves foráneas en sistemas IoT, ya que los sensores generarán millones de registros.
numeric	dht22_data, gy50_data, hcsr04_data	Almacena números con decimales exactos. Vital para la precisión de los sensores (temperatura, humedad, distancia y giroscopio).
integer	gy50_data, hcsr04_data	Entero estándar (4 bytes). Se usa para lecturas en crudo del hardware que no devuelven decimales, como el tiempo_echo o los valores raw del giroscopio.

Tipo de Dato	Tablas donde se usa	Propósito en el modelo
character varying	usuarios, auditoria	Cadenas de texto de longitud variable (VARCHAR). Usado para nombres de usuario, hashes de contraseña y nombres de acciones/tablas.
text	auditoria	Cadena de texto de longitud ilimitada. Perfecto para la columna descripcion, donde el detalle del log puede ser largo e impredecible.
boolean	usuarios	Dato lógico (verdadero/falso). Usado en activo para habilitar o deshabilitar usuarios sin borrar sus registros (Soft Delete).
date, time, timestamp	Todas las tablas	Manejo del tiempo. Se separan fecha y hora en los sensores, además de tener un timestamp general (created_at). Facilita consultas por rangos de días u horas.

Restricciones de integridad

Las restricciones garantizan la integridad, exactitud y confiabilidad de los datos dentro de la base de datos:

1. Llaves Primarias (PRIMARY KEY)

Implementación: CONSTRAINT nombre_tabla_pkey PRIMARY KEY (id) en todas las tablas. Garantiza que cada registro sea único y no nulo. PostgreSQL crea automáticamente un índice B-tree sobre esta columna, acelerando las consultas del dashboard en tiempo real.

2. Restricciones de No Nulidad (NOT NULL)

Aplicado a campos críticos como username, password_hash, temperatura, humedad, gyro_x/y/z y tiempo_echo. Evita que se inserten filas incompletas; la base de datos rechazará cualquier lectura de sensor vacía.

3. Valores por Defecto (DEFAULT)

- **Autoincremento de IDs:** `DEFAULT nextval('..._id_seq'::regclass)` — PostgreSQL gestiona el incremento automático.
- **Marcas de tiempo:** `DEFAULT CURRENT_TIMESTAMP, CURRENT_DATE, CURRENT_TIME` — La base de datos sella el momento exacto de llegada del dato.
- **Lógica de negocio:** `DEFAULT 'user'` para el rol y `DEFAULT true` para usuarios activos, facilitando la creación de usuarios sin especificar estos campos.

4. Unicidad (UNIQUE)

Implementación: `UNIQUE` en la columna `username` de la tabla `usuarios`. Impide que dos usuarios se registren con el mismo nombre, fundamental para el sistema de autenticación.

5. Llaves Foráneas (FOREIGN KEY)

Implementación: `CONSTRAINT auditoria_usuario_id_fkey FOREIGN KEY (usuario_id) REFERENCES public.usuarios(id)`. Mantiene la integridad referencial: no es posible registrar una acción de auditoría a nombre de un usuario inexistente.

6. Seguridad (RLS y políticas)

Al crear la base de datos en Supabase se activó Row Level Security (RLS) para cada tabla. Esto añade una capa de verificación a nivel de base de datos, independiente de la lógica del servidor Node.js: incluso si existiera una vulnerabilidad en el backend, los datos no serían accesibles sin la autorización correcta definida en las políticas de Supabase.

4. Integración mediante API REST

4.1 Uso de Web Sockets

La comunicación entre el ESP32 y el servidor se realiza mediante una API RESTful. El protocolo HTTP fue elegido por su compatibilidad directa con la librería `urequests` de MicroPython y su simplicidad de implementación. La siguiente tabla resume todos los endpoints disponibles:

Método	Endpoint	Descripción	Respuesta
WS	/ws	Recibe datos del ESP32 en tiempo real	—
WS	/ws-dashboard	Conexión del navegador para actualizaciones en vivo	—
POST	/api/iot/stream	Fallback HTTP si WebSocket del ESP32 falla	202 Accepted
GET	/api/realtime-state	Último estado conocido de todos los sensores	200 OK
GET	/api/dashboard-data	Datos históricos para el dashboard	200 OK

4.2 Inserción de Datos (POST)

El endpoint principal de recepción de datos es el WebSocket /ws. El ESP32 mantiene una conexión persistente y envía cada 200ms un JSON consolidado con las lecturas de los tres sensores. El endpoint POST /api/iot/stream existe únicamente como fallback HTTP en caso de que la conexión WebSocket no esté disponible. El payload es idéntico en ambos casos:

```
{
  "dht22": { "temperatura": 22.5, "humedad": 58.3 },
  "gy50": { "gyro_x": 0.12, "gyro_y": -0.05, "gyro_z": 0.33 },
  "hcsr04": { "distancia": 45.2, "distancia_izq": 38.1, "distancia_der": 52.7 }
```


}

4.3 Lectura de Datos (GET)

El endpoint GET /api/realtime-state devuelve el último estado conocido de todos los sensores desde la caché en memoria del servidor. Es consumido por el dashboard al iniciar la vista, asegurando que el usuario vea datos actuales antes del primer evento SSE.

4.4 Manejo de Errores

Cada respuesta HTTP indica el resultado de la operación con un código semántico. Todos los errores quedan registrados automáticamente en la tabla auditoria de la base de datos:

Código HTTP	Significado	Cuándo ocurre
202 Accepted	Datos recibidos y encolados	Lectura de sensor procesada correctamente
400 Bad Request	Dato malformado	El ESP32 envía un valor no numérico o JSON inválido
401 Unauthorized	Sin credenciales	Petición sin token de autenticación válido
500 Internal Error	Error de servidor	Fallo inesperado; queda registrado en auditoría

5. Desarrollo en MicroPython

5.1 Configuración de Red

El firmware del ESP32 utiliza la librería network de MicroPython para establecer la conexión Wi-Fi al inicio del sistema. El módulo se configura en modo WLAN Station (STA_IF) y se conecta a la red especificada mediante las credenciales almacenadas en el archivo de

configuración. El código implementa un bucle de espera con timeout de 10 segundos: si la conexión no se establece, el dispositivo realiza un soft reset automático. Una vez conectado, la dirección IP asignada por DHCP se imprime en consola para verificación. Tras establecer la conexión Wi-Fi, el firmware inicia una conexión WebSocket persistente con el servidor Node.js en `ws://IP:4000/ws`, siguiendo el protocolo RFC 6455. Si la conexión WebSocket falla, el sistema intenta reconectarse automáticamente antes de recurrir al fallback HTTP (`POST /api/iot/stream`).

5.2 Lectura de Datos de Sensores

El ESP32 lee tres sensores de manera periódica cada 2 segundos:

Sensor DHT22: Se utiliza la librería `dht` de MicroPython. La lectura se realiza sobre el pin GPIO configurado y retorna dos valores de punto flotante: temperatura en grados Celsius (rango -40°C a 80°C con precisión de $\pm 0.5^{\circ}\text{C}$) y humedad relativa en porcentaje (0-100% con precisión de $\pm 2\%$). Si el sensor no responde, se captura la excepción `OSError` y se registra el evento para su posterior envío como alerta al servidor.

Sensor GY-50 (L3G4200D): Se comunica con el ESP32 mediante el protocolo I2C (SDA en GPIO 21, SCL en GPIO 22). La librería de giroscopio lee los registros de los tres ejes (X, Y, Z) en velocidad angular ($^{\circ}/\text{s}$), lo que permite determinar la orientación y detección de vibraciones del vehículo. Los valores raw se convierten a grados por segundo aplicando el factor de escala del registro `CTRL_REG4`.

Sensor HC-SR04: La medición de distancia se realiza mediante pulsos digitales en los pines Trig y Echo. El firmware envía un pulso de 10 microsegundos al pin Trig y mide el tiempo de respuesta en Echo. La distancia se calcula como: $d = (\text{tiempo_echo} * \text{velocidad_sonido}) / 2$, donde $\text{velocidad_sonido} = 34300 \text{ cm/s}$. El rango operativo útil es de 2 cm a 400 cm. Distancias menores a 20 cm activan una alerta de colisión visualizada en el dashboard.

5.3 Envío de Datos

Una vez recopiladas las lecturas de los tres sensores, el firmware construye un diccionario Python que serializa a JSON y lo transmite al servidor mediante la conexión WebSocket persistente en `ws://IP:4000/ws` cada 200ms. El uso de WebSocket elimina el overhead de

establecer una nueva conexión TCP por ciclo y reduce la latencia al mínimo. Como respaldo, si el WebSocket no está disponible, el firmware puede utilizar el endpoint REST POST /api/iot/stream mediante la librería urequests. En ambos casos el payload consolidado transporta las lecturas de los tres sensores en una sola transmisión.

5.4 Manejo de Errores y Reconexión

El firmware implementa un sistema de reintentos para gestionar fallos de conectividad. Si la petición HTTP falla (excepción OSError o timeout), el código intenta reconectarse a la red Wi-Fi hasta tres veces antes de ejecutar un machine.reset() completo. Este mecanismo garantiza que el vehículo se recupere automáticamente de caídas de red sin intervención humana, lo cual es crítico para su operación autónoma. Los datos que no pudieron enviarse se descartan para evitar acumulación en memoria.

6. Sistema de Autenticación

6.1 Registro e Inicio de Sesión

El sistema de autenticación de Drako Control Center se basa en credenciales almacenadas en la tabla usuarios de PostgreSQL. Las contraseñas nunca se guardan en texto plano: se utiliza la librería bcryptjs con un factor de coste de 10 rondas para generar un hash seguro en el momento del registro. Durante el inicio de sesión, bcryptjs.compare() coteja la contraseña ingresada contra el hash almacenado. El proceso completo ocurre en el backend, por lo que el cliente nunca recibe ni transmite contraseñas en ninguna etapa. Cada intento de login exitoso o fallido queda registrado en la tabla auditoria con su timestamp y dirección IP.

6.2 Gestión de Sesiones

Una vez autenticado, el sistema persiste el estado de sesión en el cliente mediante localStorage del navegador. Se almacena un objeto JSON con los campos username, rol y un token de sesión generado por el servidor. En cada renderizado del componente React principal, se verifica la existencia y validez de este token: si no existe o ha expirado, el usuario es redirigido automáticamente a la pantalla de login. Esta validación en el cliente, complementada con la verificación en el servidor en cada petición protegida, garantiza que los datos del dashboard no sean accesibles sin autenticación previa.

6.3 Seguridad de Acceso y Roles

El sistema implementa dos roles de acceso diferenciados. El rol usuario tiene acceso de solo lectura al dashboard principal con todas las visualizaciones de sensores en tiempo real. El rol administrador accede adicionalmente a la ruta `/admin`, donde puede gestionar otros usuarios (crear, habilitar, deshabilitar) mediante el campo activo de la tabla usuarios (soft delete). La política de Row Level Security (RLS) activada en Supabase añade una capa adicional: cada operación sobre las tablas es verificada por las políticas definidas directamente en la base de datos, de forma independiente a la lógica del backend. Esto garantiza que incluso si existiera una vulnerabilidad en el servidor, los datos no serían accesibles sin la autorización correcta a nivel de base de datos.

7. Dashboard en Tiempo Real

7.1 Diseño del Dashboard

El dashboard Drako Control Center fue desarrollado en React 19 con Vite 7. La interfaz presenta una estructura de grid de 12 columnas con paneles modulares: cada sensor o componente ocupa una tarjeta independiente con sus valores en tiempo real, historial gráfico y estado de alerta. Los iconos provienen de Lucide React, las animaciones de Framer Motion y el diseño es completamente responsivo. El sistema está organizado en cuatro páginas: LandingPage (presentación con animaciones), Dashboard (telemetría principal), Admin (gestión de usuarios, solo rol administrador) y Profile (edición de perfil de usuario).

7.2 Visualización de Datos

El dashboard incluye nueve widgets distribuidos en el grid de 12 columnas. Cada uno utiliza una tecnología de visualización diferente, seleccionada según la naturaleza del dato que representa:

Widget / Sensor	Tipo de visualización	Descripción
Barra de estado del robot	Barra de estado dinámica (ancho completo)	Infiere el estado de navegación a partir de las distancias en tiempo real. Estados posibles: CAMINO LIBRE, OBSTÁCULO FRONTAL, PARED IZQUIERDA, PARED DERECHA y SIN SEÑAL. Incluye indicador activo/sin señal del HC-SR04.
DHT22 · Ambiente	Tarjeta de valores + ComposedChart (Recharts)	Muestra temperatura actual en °C y humedad relativa en % con iconos. La gráfica histórica combina área para temperatura y línea para humedad. Alerta visual si temperatura supera 45°C (riesgo batería LiPo).
GY-50 · Giroscopio	Cubo 3D (CSS Transforms)	Cubo tridimensional renderizado con CSS que rota en tiempo real según los valores reales del giroscopio: Pitch (eje X), Yaw (eje Y) y Roll (eje Z). Muestra también los valores numéricos de cada eje.
HC-SR04 · Sonar ×3	Radar animado + badges de proximidad	Radar con barrido circular que muestra la distancia frontal. Tres badges (IZQ / CENTRO / DER) con alarma visual cuando la distancia es menor al umbral configurado (frontal < 20 cm, laterales < 15 cm).
Drako · Dinámica 3D	Maqueta 3D isométrica (CSS Transforms)	Representación 3D del robot vista isométrica. El chasis gira con los datos del giroscopio (pitch + roll). Las ruedas delanteras rotan $\pm 35^\circ$ según la dirección de corrección. Los faros cambian de color según el estado (blanco normal / rojo parpadeante en alerta). Los dots de sensores se vuelven rojos al detectar obstáculo cercano. El

Widget / Sensor	Tipo de visualización	Descripción
		glow del piso cambia de color según el estado de navegación.
Análisis Ambiental	ComposedChart (Recharts) — 8 columnas	Gráfica histórica combinada de temperatura y humedad con línea de tiempo configurable.
Auditoría del Sistema	Log de eventos — 4 columnas	Registro de eventos críticos provenientes de la tabla auditoria de la base de datos. Muestra acciones, tablas afectadas y timestamps.
Estabilidad Inercial GY-50	LineChart (Recharts) — 6 columnas	Gráfica de tres líneas simultáneas para los ejes X (pitch), Y (yaw) y Z (roll) del giroscopio. Permite visualizar la estabilidad del vehículo a lo largo del tiempo.
Histórico de Proximidad ×3	LineChart (Recharts) — 6 columnas	Tres líneas simultáneas que muestran el historial de distancias de los sensores izquierdo, central y derecho. Permite identificar patrones de obstáculos en la trayectoria del robot.

Actualización en tiempo real: todos los widgets reciben datos mediante WebSocket (endpoint `ws://host/ws-dashboard`). Al conectarse, el navegador recibe el último estado de todos los sensores (type: 'connected'). Cada transmisión del ESP32 genera un broadcast inmediato a todos los browsers conectados (type: 'sensor-realtime'), y cada 2 segundos se sincroniza el histórico desde la base de datos (type: 'dashboard-update').

7.3 Actualización en Tiempo Real

El dashboard mantiene una conexión WebSocket persistente con el servidor en `ws://host/ws-dashboard`. Al conectarse, el navegador recibe inmediatamente el último estado de todos los sensores (type: 'connected'). Cada vez que el ESP32 envía datos, el servidor hace broadcast instantáneo a todos los browsers conectados (type:

'sensor-realtime'). Adicionalmente, cada 2 segundos se sincroniza el histórico de la base de datos y se distribuye a todos los clientes (type: 'dashboard-update'). El servidor detecta y limpia conexiones de navegador muertas mediante un mecanismo de ping/pong cada 30 segundos. No se utiliza SSE ni EventSource en ninguna parte del sistema.

7.4 Interactividad y Manejo de Errores

El usuario puede filtrar el rango temporal de los gráficos históricos y mostrar u ocultar sensores individuales. Si el servidor no reporta datos por más de 30 segundos, los paneles afectados muestran un indicador de advertencia visual sin interrumpir el resto del dashboard.

8. Conclusiones

8.1 Resultados Obtenidos

La arquitectura adoptada utiliza WebSocket en ambos extremos: el ESP32 transmite datos al servidor cada 200ms vía `ws://IP:4000/ws`, y el navegador recibe actualizaciones en vivo vía `ws://host/ws-dashboard`. Esto elimina la latencia y el overhead asociados a HTTP y SSE, logrando un pipeline de datos completamente basado en conexiones persistentes. El HC-SR04 detectó correctamente los tres vectores de distancia (izquierda, centro, derecha), permitiendo la activación de las alertas de proximidad y la representación tridimensional del estado de navegación en el widget Drako 3D.

8.2 Problemas Encontrados y Soluciones

Durante el desarrollo se presentaron tres desafíos principales. El primero fue la inestabilidad de la conexión Wi-Fi del ESP32 bajo carga alta, resuelto implementando el mecanismo de reintentos con soft reset automático descrito en la sección 5. El segundo fue la latencia en la escritura de datos a Supabase desde el servidor, solucionada migrando las operaciones de persistencia a un modelo asíncrono con promesas encadenadas (`async/await`) que no bloquea el hilo principal del servidor Node.js. El tercer desafío fue la sincronización de la animación del cubo 3D con los valores del giroscopio, resuelta normalizando los valores raw del GY-50 a un rango de -180° a 180° antes de aplicarlos a las transformaciones CSS.

8.3 Aprendizaje Logrado

El proyecto representó un ejercicio práctico integral que articuló conocimientos de electrónica embebida, desarrollo web full-stack y arquitectura de sistemas IoT. El equipo adquirió competencias en la programación de microcontroladores en MicroPython, en el diseño de APIs RESTful y esquemas de base de datos relacionales en PostgreSQL, y en la construcción de interfaces reactivas con React. Más allá de las habilidades técnicas, el proyecto demostró que la tecnología IoT accesible puede aplicarse a soluciones de impacto social concreto en el contexto boliviano, abriendo posibilidades para la expansión futura de Drako con navegación asistida por visión artificial y reconocimiento de voz en español.

9. Anexos

- Código fuente (GitHub):

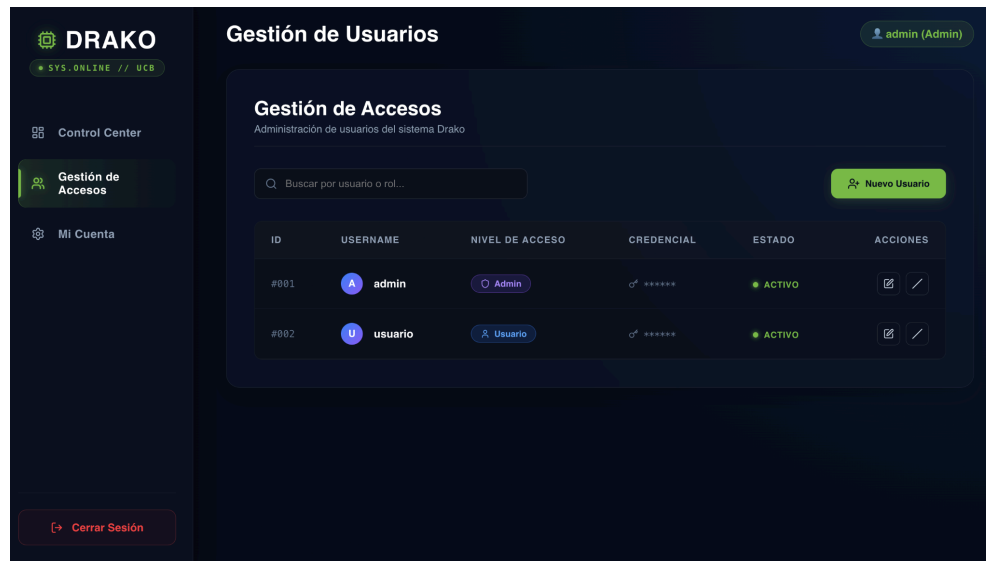
<https://github.com/ChristianCoronel404/proyecto-iot>

- Capturas:

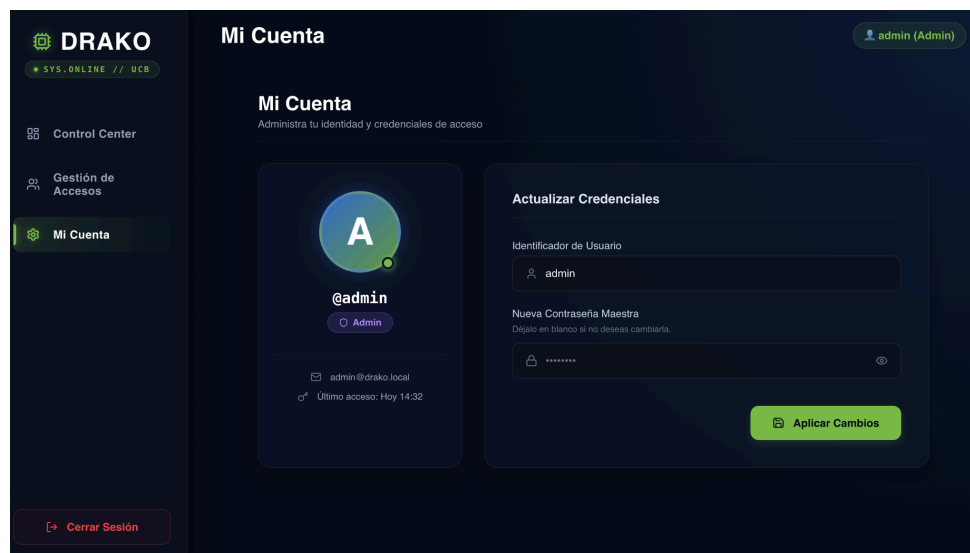
- Dashboard de control:



- Gestión de accesos:



- Mi cuenta



- Pagina principal



- Login

The screenshot shows the login form for DraKo OS. The form is titled "Drako OS" and has a close button (X) in the top right corner. It contains two input fields: "Usuario" with the placeholder text "Ej. Admin" and "Contraseña" with a masked password (dots). Below the input fields is a large green button labeled "Acceder al Sistema".

10. Referencias

- Espressif Systems. (2023). ESP32 technical reference manual (v5.2). Espressif Systems.
https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- MicroPython Project. (2024). MicroPython documentation for ESP32. MicroPython.
<https://docs.micropython.org/en/latest/esp32/>
- Aosong Electronics. (2022). DHT22 product manual: Digital-output relative humidity & temperature sensor/module AM2302. Aosong.
<https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf>
- STMicroelectronics. (2011). L3G4200D: MEMS motion sensor, three-axis digital output gyroscope (DocID 17116 Rev 3). STMicroelectronics.
<https://www.st.com/resource/en/datasheet/l3g4200d.pdf>
- Elec Freaks. (2020). Ultrasonic ranging module HC-SR04 user guide. Elec Freaks.
<https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>
- OpenJS Foundation. (2024). Node.js documentation (v20 LTS). OpenJS Foundation.
<https://nodejs.org/en/docs/>
- Luber, T., & contributors. (2024). Express.js 4.x API reference. OpenJS Foundation.
<https://expressjs.com/en/4x/api.html>
- The PostgreSQL Global Development Group. (2024). PostgreSQL 16 documentation. The PostgreSQL Global Development Group. <https://www.postgresql.org/docs/16/>
- Supabase Inc. (2024). Supabase documentation: Row level security. Supabase.
<https://supabase.com/docs/guides/auth/row-level-security>
- Meta Platforms, Inc. (2024). React: A JavaScript library for building user interfaces (v18). Meta. <https://react.dev/reference/react>
- recharts contributors. (2024). Recharts: Redefined chart library built with React and D3 (v2.12). GitHub. <https://recharts.org/en-US/>

kelektiv. (2023). bcryptjs: Optimized bcrypt in JavaScript with zero dependencies (v2.4.3). npm. <https://www.npmjs.com/package/bcryptjs>

MDN Web Docs. (2024). Server-Sent Events: Using server-sent events. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

Kicad EDA. (2023). KiCad EDA 7.0: Schematic capture and PCB layout documentation. KiCad. <https://docs.kicad.org/>

Chiarella, S., & contributors. (2024). Fritzing: Prototyping tools for makers (v1.0). Fritzing. <https://fritzing.org/>